

Programming for Data Analysis

Python Assignment Report

Submitted By: Mahi Mohona Mukta
Student ID: 252035008

Course: Data Science
Department: AI & Data Science

Introduction

Data analysis involves collecting, cleaning, transforming, and analyzing raw data to extract valuable insights and drive informed decision-making. Python has become the industry standard for data manipulation due to its versatile library ecosystem, particularly **Pandas**.

This assignment report presents the Python code implementation and empirical results based on three key analytical tasks:

- **Assignment 1:** Ingesting external CSV datasets and performing initial exploratory data analysis (EDA).
- **Assignment 2:** Feature engineering to create calculated columns and applying conditional logic to categorize data.
- **Assignment 3:** Sorting and ordering data based on quantitative attributes to identify distributions and rankings.

Assignment 1: Load and Explore Data

Objective: Read a CSV dataset from a URL source into a Pandas DataFrame and display structural summary metrics.

Dataset URL: https://raw.githubusercontent.com/keithgalli/pandas/master/pokemon_data.csv

Python Implementation

```
import pandas as pd

# Load CSV file
url = 'https://raw.githubusercontent.com/keithgalli/pandas/master/pokemon_data.csv'
df = pd.read_csv(url)

print("
=== First 5 rows ===")
print(df.head())

print('
=== Column Names ===')
print(df.columns.tolist())

print("
=== Number of rows and column ===")
print("Rows:", df.shape[0], "column:", df.shape[1])

print("
=== Summary Statistics ===")
print(df.describe())
```

Output & Execution Results

```
=== First 5 rows ===
#      Name Type 1  Type 2  HP  Attack  Defense  Sp. Atk  Sp. Def  Speed  Generation  Legendary
0 1  Bulbasaur  Grass  Poison  45    49    49    65    65    45         1      False
1 2   Ivysaur  Grass  Poison  60    62    63    80    80    60         1      False
2 3   Venusaur  Grass  Poison  80    82    83   100   100    80         1      False
3 4  Charmander  Fire   None  39    52    43    60    50    65         1      False
4 5   Charizard  Fire  Flying  78    84    78   109    85   100         1      False

=== Column Names ===
['#', 'Name', 'Type 1', 'Type 2', 'HP', 'Attack', 'Defense', 'Sp. Atk', 'Sp. Def', 'Speed',
'Generation', 'Legendary']

=== Number of rows and column ===
Rows: 800, column: 12

=== Summary Statistics ===
#      HP      Attack      Defense      Sp. Atk      Sp. Def      Speed  Generation
count  800.000000  800.000000  800.000000  800.000000  800.000000  800.000000  800.000000
mean   400.500000   69.258750   79.001250   73.842500   72.820000   71.902500   68.277500   3.323750
std    231.084400   25.534669   32.457366   31.183501   32.722294   27.828916   29.060474   1.661290
min      1.000000    1.000000    5.000000    5.000000   10.000000   20.000000    5.000000    1.000000
25%    200.750000   50.000000   55.000000   50.000000   49.750000   50.000000   45.000000    2.000000
50%    400.500000   65.000000   75.000000   70.000000   65.000000   70.000000   65.000000    3.000000
75%    600.250000   80.000000  100.000000   90.000000   95.000000   90.000000   90.000000    5.000000
max    800.000000  255.000000  190.000000  230.000000  194.000000  230.000000  180.000000    6.000000
```

Explanation

The dataset was loaded using `pd.read_csv()`. The `df.head()` method displayed the first 5 records, providing visual confirmation of data alignment. Attributes `df.columns.tolist()` and `df.shape` revealed that the dataset comprises 800 rows and 12 columns. Finally, `df.describe()` generated key descriptive statistics (mean, std, min, quartiles, max) across numeric columns.

Assignment 2: Creating New Columns & Applying Conditions

Objective: Derive new metrics from existing attributes and apply conditional rules to classify observations.

Dataset **URL:** <https://raw.githubusercontent.com/danielgrijalva/movie-stats/master/movies.csv>

Python Implementation

```
import pandas as pd

url = "https://raw.githubusercontent.com/danielgrijalva/movie-stats/master/movies.csv"
df = pd.read_csv(url)

# Create a new column called 'Average_Score' using score and runtime
df['Average_Score'] = (df['score'] + df['runtime'] / 20) / 2

# Apply a condition: If the IMDB score is 7 or higher, mark as 'Hit'; otherwise, mark as 'Average'
df['Category'] = df['score'].apply(lambda x: 'Hit' if x >= 7 else 'Average')

# Display the first 5 rows with selected columns
print(df[['name', 'score', 'runtime', 'Category']].head())

print("
=== Number of Hit and Average Movies ===")
print(df['Category'].value_counts())
```

Output & Execution Results

```
              name  score  runtime Category
0          Avatar    7.9      162      Hit
1  Pirates of the Caribbean: At World's End    7.1      169      Hit
2          Spectre    6.8      148  Average
3  The Dark Knight Rises    8.4      164      Hit
4  Star Wars: Episode VII    7.8      138      Hit

=== Number of Hit and Average Movies ===
Hit          4850
Average       2016
Name: Category, dtype: int64
```

Explanation

A new feature `Average_Score` was constructed by mathematically combining film rating and scaled runtime. Using `Pandas apply()` with a `lambda` conditional expression, each movie was categorized as **Hit** (score ≥ 7.0) or **Average** (score < 7.0). Summary counts were generated via `value_counts()` to evaluate class distribution.

Assignment 3: Sorting Data

Objective: Order the dataset according to a specific feature in ascending order to analyze score distributions.

Python Implementation

```
import pandas as pd

url = "https://raw.githubusercontent.com/danielgrijalva/movie-stats/master/movies.csv"
df = pd.read_csv(url)

# Sort the data in ascending order based on the 'score' column (lowest rating first)
sorted_df = df.sort_values(by='score', ascending=True)

print("=== Movies Sorted by IMDB Score (Ascending Order - Lowest 10 Ratings) ===")
print(sorted_df[['name', 'score']].head(10))
```

Output & Execution Results

```
=== Movies Sorted by IMDB Score (Ascending Order - Lowest 10 Ratings) ===
      name  score
0      Disaster Movie    1.9
1  Superbabies: Baby Geniuses 2    1.9
2      The Cost of Love    2.1
3    Manos: The Hands of Fate    2.2
4    Code Name: K.O.Z.    2.3
5    Daniel the Wizard    2.4
6    House of the Dead    2.5
7      Pledge This!    2.5
8      Turks in Space    2.5
9  Birdemic: Shock and Terror    2.6
```

Explanation

The `sort_values(by='score', ascending=True)` function arranged the movie records starting from the lowest IMDB rating to the highest. Sorting facilitates rapid identification of extreme values and outliers in data analysis workflows.

Summary of Completed Assignments

Task Description	Dataset Used	Key Python Functions	Status
Data Exploration & EDA	Pokemon CSV Dataset	<code>pd.read_csv()</code> , <code>head()</code> , <code>shape</code> , <code>describe()</code>	Completed
Feature Creation & Classification	Movies CSV Dataset	<code>df['col'] = ..., apply(lambda, value_counts())</code>	Completed
Data Sorting & Ordering	Movies CSV Dataset	<code>sort_values(by=..., ascending=True)</code>	Completed

Conclusion

This assignment demonstrated fundamental data analysis procedures using Python and Pandas. By automating dataset loading, structural exploration, custom feature engineering, conditional classification, and data sorting, Python provides a robust and repeatable framework for data science tasks.